GUIA VISUAL 017

LOCALIZANDO ARQUIVOS

"Quais arquivos de 100 MB mudaram no mês?" e "onde está o `nmap`?" são perguntas de ferramentas diferentes.

017
GUIAS LINUX

A LINHA DO `FIND` : QUATRO PARTES, ORDEM FIXA

ONDE	OPÇÕES GLOBAIS	TESTES	AÇÃO
`/var/log`	`-maxdepth 2 -xdev`	`-type f -size +10M`	`-exec ls -lh {} +`
ponto de partida da descida	valem para a busca inteira	um **E** com curto-circuito: o 1º que falha descarta	só no que passou em todos os testes

Teste barato primeiro é otimização real. E a regra de escolha entre as famílias: **locate** para "onde fica um arquivo com esse nome?" · **find** para tamanho, data, dono, permissão · **type** para comandos.

OS TESTES QUE SE USA DE VERDADE

NOME E CAMINHO

- `-name "*.conf"` — só o nome-base, com glob
- `-iname "*.CONF"` — ignora maiúsculas
- `-path "*/etc/*.conf"` — casa o caminho inteiro
- `-regex '.*cap0[12].*'` — regex sobre o caminho

TIPO E TAMANHO

- `-type f · d · l` — arquivo, diretório, link
- `-xtype l` — link **quebrado**
- `-size +100M · -1k` — sem sufixo = blocos de 512 B
- `-empty` — arquivo ou diretório vazio

TEMPO

- `-mtime -7` — conteúdo mudou em 7 dias
- `-mmin -15` — nos últimos 15 minutos
- `-cmin -60` — o **inode** mudou na última hora
- `-newer ref.txt` — mais recente que outro

DONO E PERMISSÃO

- `-user kali -group sudo` — dono e grupo
- `-perm 644` — exatamente isso
- `-perm -u+w` — pelo menos esse bit
- `-perm /o+w` — qualquer bit para outros

O sinal muda tudo: sem sinal é valor *exato*, `+` é "mais que", `-` é "menos que" — vale para `-size`, `-mtime`, `-links` e `-perm`. E `-exec … +` agrupa numa execução; `-exec … \;` abre um processo por arquivo.

ONDE ESTÁ UM COMANDO — E NA PRÁTICA

ORDEM DE RESOLUÇÃO DO SHELL

1. `alias`
2. `função do shell`
3. `builtin`
4. `tabela hash`
5. `varredura do PATH`

✕ `command not found`

`type -a` vê as cinco · **`command -v`** idem, forma POSIX para script · **`which`** é programa externo e só enxerga o **`PATH`**: ignora alias, função e builtin — os três lugares onde um comando pode ter sido sequestrado.

```
kali@kali: ~
$ find /var/log -xdev -type f -size +10M \
    -mtime -7 -printf '%s\t%p\n' | sort -rn | head
$ find . -name node_modules -prune -o -type f -print
# -prune so funciona no idioma "-o ... -print"
$ find . -type f -print0 | xargs -0 grep -l ERRO
# -print0/-0 sempre: nome pode ter espaco
$ locate -b '\rockyou.txt'
/usr/share/wordlists/rockyou.txt.gz
$ type -a ls
ls is aliased to `ls --color=auto'
ls is /usr/bin/ls
```

Busca destrutiva ensaia antes. Rode a expressão com `-print`, confira a lista e só então troque pela ação. O `-delete` implica `-depth` e muda a ordem de avaliação — combinado com `-prune` já produziu remoção fora do alvo. Sem parênteses, `find . -name a -o -name b -delete` apaga só os `b`. Em expressão complexa, prefira `-print0 | xargs -0 rm --` depois de ler a lista.

RESUMO RÁPIDO

- ✓ `find` varre a árvore; `locate` consulta um índice que envelhece (`sudo updatedb`).
- ✓ `-name` vê o nome-base; `-path` vê o caminho inteiro.
- ✓ `-xdev` é reflexo em busca a partir de `/` : evita `/proc` , `/sys` e discos montados.
- ✓ `type` no terminal, `command -v` em script — `which` não vê alias nem função.
- ✓ Binário recém-instalado "não existe"? `hash -r` limpa a memória de caminhos do shell.

NO KALI

O `locate` não vem instalado: `sudo apt install plocate && sudo updatedb` — e achar wordlist é caso dele, não de `find / .` Inspeção do **próprio** sistema em uma linha: `find / -xdev -perm -4000 -type f 2>/dev/null` (SUID) e `-perm /o+w` (gravável por qualquer um); em máquina de terceiros, só com autorização escrita. E `type -a ls` denuncia o alias malicioso que o `which` jamais mostraria.